JavaScript ES6+ & jQuery Cheat Sheet

ES6+ VS JQUERY CORE

Guía de Referencia Frontend de Nivel Senior y Migración Legacy

1. Fundamentos Avanzados de JavaScript Moderno (ES6+)

Motor JS: Scope, Closures y Prototype

- **Closures:** Funciones que recuerdan su ámbito léxico original incluso cuando son ejecutadas fuera de él. Base de la privacidad de variables.
- **Prototypal Inheritance:** En JS los objetos heredan propiedades directamente de otros objetos a través de la cadena `__proto__`.
- **Event Loop en Navegador:** Call Stack → Microtasks (Promises, MutationObserver) → Macrotasks (setTimeout, DOM events, AJAX).

Asincronía & ES6+ Features

- **Async/Await & Promises:** Manejo elegante del flujo asíncrono sobre la API de Promesas (`Promise.all()`, `Promise.race()`).
- **Destructuring & Rest/Spread:** Extraer valores de arrays/objetos de forma limpia (`const { name, ...rest } = obj`).
- **ES Modules:** `import / export` nativo para estructurar aplicaciones modulares sin globales.

2. Manipulación del DOM y Eventos: JavaScript Vanilla vs jQuery

Selección y Manipulación del DOM

- **Seleccionar:**
  - Vanilla: `document.querySelector('.card')`
  - jQuery: `$('.card')`
- **Modificar Contenido / Atributos:**
  - Vanilla: `el.textContent = 'Hola'` | `el.setAttribute('id', 'x')`
  - jQuery: `$el.text('Hola')` | `$el.attr('id', 'x')`
- **Manipulación de Clases:**
  - Vanilla: `el.classList.add('active')` / `remove()`
  - jQuery: `$el.addClass('active')` / `removeClass()`

Eventos y Delegación de Eventos

- **Event Listeners Directos:**
  - Vanilla: `el.addEventListener('click', fn)`
  - jQuery: `$el.on('click', fn)`
- **Delegación de Eventos (Elementos Dinámicos):**
  - Vanilla: `parent.addEventListener('click', e => { if (e.target.matches('.item')) ... })`
  - jQuery: `$(parent).on('click', '.item', fn)`
- **DOM Ready:**
  - Vanilla: `document.addEventListener('DOMContentLoaded', fn)`
  - jQuery: `$(function() { ... })`

3. Peticiones HTTP Asíncronas y Estrategias de Migración

Diferenciador Senior: En aplicaciones modernas, `fetch()` o `axios` reemplazan por completo a `$.ajax()`. Al mantener proyectos legacy, abstrae las llamadas jQuery dentro de funciones que retornen Promesas nativas de JS para facilitar una futura migración hacia frameworks modernos (Vue, React).

AJAX: jQuery $.ajax vs Fetch API

- **jQuery AJAX:** `$.ajax({ url, method, data }).done(fn)`. Basado en objetos Deferred/jqXHR propios.

Performance: Vanilla vs jQuery

- El DOM nativo de Vanilla JS es drásticamente más rápido al evitar los envoltorios (*wrappers*) y traversals del objeto jQuery.

- **Fetch API (Vanilla):** `const res = await fetch(url); const data = await res.json();`. Nativo en todos los navegadores modernos.

- Usa jQuery únicamente para mantener código heredado (legacy) o plugins específicos de su ecosistema.

4. Ejercicio Práctico: Comparativa Directa Vanilla JS (ES6+) vs jQuery

```
// ============================================================
// Opción A: JavaScript Vanilla Moderno (Recomendado)
// ============================================================
document.addEventListener('DOMContentLoaded', () => {
  const btn = document.querySelector('#fetch-btn');
  const list = document.querySelector('#user-list');

  btn?.addEventListener('click', async () => {
    try {
      const res = await fetch('https://api.example.com/users');
      const users = await res.json();

      list.innerHTML = users.map(u => `<li class="user-item">${u.name}</li>`).join('');
    } catch (err) {
      console.error('Error cargando usuarios:', err);
    }
  });
});

// ============================================================
// Opción B: Formato Equivalente con jQuery (Legacy Pattern)
// ============================================================
$(function() {
  $('#fetch-btn').on('click', function() {
    $.getJSON('https://api.example.com/users')
      .done(function(users) {
        const items = users.map(u => `<li class="user-item">${u.name}</li>`);
        $('#user-list').html(items.join(''));
      })
      .fail(function(xhr, status, err) {
        console.error('Error cargando usuarios:', err);
      });
  });
});
```